Book: Essential Matlab for Engineers and Scientists, by Hahn and Valentine, page 334-344

In certain problems, `diff` is helpful in finding approximate derivatives. For example, if `x` contains displacements of an object every `h` seconds, `diff(x)/h` is its speed.

17.4 FIRST-ORDER DIFFERENTIAL EQUATIONS

The most interesting real-life situations that we may want to model, or represent quantitatively, are usually those in which the variables change in time (e.g., biological, electrical, or mechanical systems). If the changes are continuous, a system can often be represented with equations involving the derivatives of the dependent variables. Such equations are referred to as *differential*. The frequent aim of modeling is to be able to write down a set of differential equations (DEs) that describe the system being studied as accurately as possible. Very few DEs can be solved analytically, so numerical methods are once again required. We consider the simplest one in this section: Euler's (rhymes with "boiler"). We also consider briefly how to improve it.

17.4.1 Euler's method

In general we want to solve a first-order DE (strictly, an ordinary DE, or ODE) of the form

$$dy/dx = f(x, y), \quad y(0) \text{ given}$$

Euler's method consists of numerically replacing dy/dx with its Newton quotient so that the ODE becomes

$$\frac{y(x+h) - y(x)}{h} = f(x, y)$$

After a slight rearrangement of terms, we get

$$y(x+h) = y(x) + hf(x, y) \tag{17.6}$$

Solving an ODE numerically is such an important and common problem in science and engineering that it is worth introducing some general notation at this point. Suppose we want to integrate the ODE over the interval $x = a$ ($a = 0$ usually) to $x = b$. We break this interval into m steps of length h, so

$$m = (b - a)/h$$

(this is the same as the notation used in the update process of Chapter 11, except that dt has been replaced by the more general h).

For consistency with MATLAB's subscript notation, if we define y_i as $y(x_i)$ (the Euler estimate at the *beginning* of step i), where $x_i = (i - 1)h$, then

$y_{i+1} = y(x + h)$ at the end of step i. We can then replace Equation (17.6) by the iterative scheme

$$y_{i+1} = y_i + hf(x_i, y_i) \tag{17.7}$$

where $y_1 = y(0)$.

17.4.2 Example: Bacteria colony growth

Suppose a colony of 1000 bacteria is multiplying at the rate of $r = 0.8$ per hour per individual (i.e., an individual produces an average of 0.8 offspring every hour). How many bacteria are there after 10 hours? Assuming that the colony grows continuously and without restriction, we can model this growth with the DE

$$dN/dt = rN, \quad N(0) = 1000 \tag{17.8}$$

where $N(t)$ is the population size at time t. This process is called *exponential growth*, and Equation (17.8) may be solved analytically to give the well-known formula for it:

$$N(t) = N(0)e^{rt}$$

To solve Equation (17.8) numerically, we apply Euler's algorithm to get

$$N_{i+1} = N_i + rhN_i \tag{17.9}$$

where the initial value $N_1 = 1000$.

It is very easy to program Euler's method. The following script implements Equation (17.9), taking $h = 0.5$. It also computes the exact solution for comparison.

```
h = 0.5;
r = 0.8;
a = 0;
b = 10;
m = (b - a) / h;
N = zeros(1, m+1);
N(1) = 1000;
t = a:h:b;

for i = 1:m
  N(i+1) = N(i) + r * h * N(i);
end
```

```
Nex = N(1) * exp(r * t);
format bank
disp( [t' N' Nex'] )

plot(t, N ), xlabel( 'Hours' ), ylabel( 'Bacteria' )
hold on
plot(t, Nex ), hold off
```

Results are shown in Table 17.1, and in Figure 17.5. The Euler solution is not good. In fact, the error gets worse at each step, and after 10 hours of bacteria time it is about 72%. We can improve it if we make h smaller, but there will always be some value of t where the error exceeds an acceptable limit.

Euler's method performs better in some cases than it does here, but other numerical methods always do better. Two of them are discussed below. More sophisticated methods may be found in most textbooks on numerical analysis; still, Euler's method may always be used as a first approximation as long as you realize that errors may arise.

17.4.3 Alternative subscript notation

Equation 17.9 is a *finite difference scheme*. The conventional finite difference notation is for the initial value to be represented by N_0—that is, with subscript $i = 0$. N_i is then the estimate at the *end* of step i. If you want the MATLAB subscripts in the Euler solution to be the same as the finite difference subscripts, the initial value N_0 must be represented by the MATLAB scalar `N0`, and you must compute `N(1)` separately, before the `for` loop starts. You also have to display

Table 17.1 Bacteria Colony Growth

Time (hours)	Euler	Predictor-Corrector	Exact
0.0	1000	1000	1000
0.5	1400	1480	1492
1.0	1960	2190	2226
1.5	2744	3242	3320
2.0	3842	4798	4953
⋮			
5.0	28,925	50,422	54,598
⋮			
8.0	217,795	529,892	601,845
⋮			
10.0	836,683	2,542,344	2,980,958

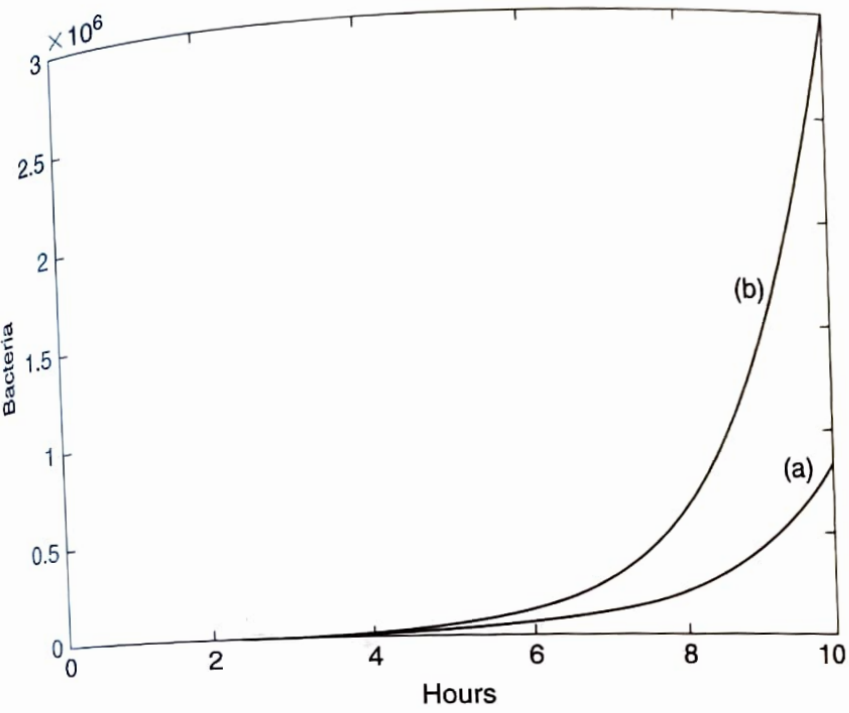


FIGURE 17.5
Bacteria colony growth: (a) Euler's method; (b) exact solution.

or plot the initial values separately since they will no longer be included in the MATLAB vectors `t`, `N`, and `Nex` (which now have m instead of $m + 1$ elements). Here is a complete script to generate the Euler solution using finite difference subscripts:

```
h = 0.5;
r = 0.8;
a = 0;
b = 10;
m = (b - a) / h;
N = zeros(1, m);      % one less element now
N0 = 1000;
N(1) = N0 + r*h*N0;   % no longer 'self-starting'

for i = 2:m
 N(i) = N(i-1) + r * h * N(i-1); %finite difference notation
end

t = a+h:h:b;           % exclude initial time = a
Nex = N0 * exp(r * t);
disp( [a N0 N0] )      % display initial values separately
disp( [t' N' Nex'] )
```

```
plot(a, N0)          % plot initial values separately
hold on
plot(t, N ), xlabel( 'Hours' ), ylabel( 'Bacteria' )
plot(t, Nex ), hold off
```

17.4.4 A predictor-corrector method

One improvement on the numerical solution of the first-order DE

$$dy/dx = f(x, y), \quad y(0) \text{ given}$$

is as follows. The Euler approximation, which we are going to denote by an asterisk, is given by

$$y^*_{i+1} = y_i + hf(x_i, y_i) \tag{17.10}$$

but it favors the old value of y in computing $f(x_i, y_i)$ on the right-hand side. Surely it would be better to say

$$y^*_{i+1} = y_i + h\left[f(x_{i+1}, y^*_{i+1}) + f(x_i, y_i)\right]/2 \tag{17.11}$$

where $x_{i+1} = x_i + h$, since this also involves the new value y^*_{i+1} in computing f on the right-hand side. Of course, the problem is that y^*_{i+1} is as yet unknown, so we can't use it on the right-hand side of Equation (17.11). But we could use Euler to estimate (predict) y^*_{i+1} from Equation (17.10) and then use Equation (17.11) to correct the prediction by computing a *better* version of y^*_{i+1}, which we will call y_{i+1}. The full procedure is thus

Repeat as many times as required:
Use Euler to predict: $y^*_{i+1} = y_i + hf(x_i, y_i)$
Then correct y^*_{i+1} to $y_{i+1} = y_i + h[f(x_{i+1}, y^*_{i+1}) + f(x_i, y_i)]/2$

This *predictor-corrector* method can easily be adapted to our problem. The relevant lines of code are

```
for i = 1:m    % m steps of length dt
  ne(i+1) = ne(i) + r * h * ne(i);
  np = nc(i) + r * h * nc(i);
  nc(i+1) = nc(i) + r * h * (np + nc(i))/2;
  disp( [t(i+1) ne(i+1) nc(i+1) nex(i+1)] )
end;
```

`ne` stands for the "straight" (uncorrected) Euler solution; `np` is the Euler predictor (since this is an intermediate result, a vector is not needed for `np`); and `nc` is the corrector. The worst error is now only 15%, which is much better than the uncorrected Euler solution, although there is still room for improvement.

17.5 LINEAR ORDINARY DIFFERENTIAL EQUATIONS

Linear ordinary differential equations (ODEs) with constant coefficients may be solved analytically in terms of *matrix exponentials*, which are represented in MATLAB by the function `expm`. For an example, see **MATLAB Help: Mathematics: Matrices and Linear Algebra: Matrix Powers and Exponentials**.

17.6 RUNGE-KUTTA METHODS

A variety of algorithms with the general name of Runge-Kutta can be used to integrate systems of ODEs. The formulae involved are rather complicated; they can be found in most books on numerical analysis.

However, as you may have guessed, MATLAB has plenty of ODE solvers, which are discussed in **MATLAB Help: Mathematics: Differential Equations**. Among them are `ode23` (second/third order) and `ode45` (fourth/fifth order), which implement Runge-Kutta methods. (The *order* of a numerical method is the power of h (i.e., dt) in the leading error term. Since h is generally very small, the higher the power, the smaller the error.) We will demonstrate the use of `ode23` and `ode45` here, first with a single first-order ODE and then with systems of such equations.

17.6.1 A single differential equation

Here's how to use `ode23` to solve the bacteria colony growth problem, Equation (17.8):

$$dN/dt = rN, \quad N(0) = 1000$$

Step 1 Write a function file for the *right-hand side* of the DE to be solved. The function must have input variables t and N in this case (i.e., independent and dependent variables of the DE), *in that order*. Create the function file `f.m` as follows:

```
function y = f(t, Nr)
y = 0.8 * Nr;
```

Step 2 Enter the following statements in the Command Window:

```
a = 0;
b = 10;
n0 = 1000;
[t, Nr] = ode23(@f, [a:0.5:b], n0);
```

Step 3 Note that the input arguments of `ode23`:

- `@f` is a handle for the function `f`, which contains the right-hand side of the DE;
- `[a:0.5:b]` is a vector (`tspan`) specifying the range of integration. If `tspan` has two elements (`[a b]`), the solver returns the solution evaluated at every integration step (it chooses the integration steps and may vary them). This form is suitable for plotting. However, if you want to display the solution at regular time intervals, as we want to here, use the form of `tspan` with three elements as above. The solution is then returned evaluated at each time in `tspan`. The accuracy of the solution is not affected by the form of `tspan` used.
- `n0` is the initial value of the solution N.

Step 4 The output arguments are two vectors—that is, the solutions `Nr` at times `t`. For 10 hours `ode23` gives a value of 2,961,338 bacteria. From the exact solution in Table 17.1, we see that the error here is only 0.7%.

If the solutions you get from `ode23` are not accurate enough, you can request greater accuracy with an additional optional argument. (See Help.) If you need still more accurate numerical solutions, you can use `ode45` instead, which gives a final value of 2,981,290—an error of about 0.01%.

17.6.2 Systems of differential equations: Chaos

That weather prediction is so difficult and forecasts so erratic is no longer thought to be the result of complexity of the system but rather the result of the nature of the ODEs modeling it. These ODEs belong to a class referred to as *chaotic*; they produce wildly different results when their initial conditions are changed infinitesimally. In other words, accurate weather prediction depends crucially on the accuracy of the measurements of initial conditions.

Edward Lorenz, a research meteorologist, discovered this phenomenon in 1961. His original equations are far too complex to consider here, but the following much simpler system has essentially the same chaotic features:

$$dx/dt = 10(y - x) \tag{17.12}$$

$$dy/dt = -xz + 28x - y \tag{17.13}$$

$$dz/dt = xy - 8z/3 \tag{17.14}$$

This system may be solved very easily with MATLAB's ODE solvers. The idea is to solve the ODEs with certain initial conditions, plot the solution, and then change the initial conditions very slightly and superimpose the new solution over the old one to see how much it has changed.

We begin by solving the system with the initial conditions $x(0) = -2$, $y(0) = -3.5$ and $z(0) = 21$.

Step 1 Write a function file `lorenz.m` to represent the right-hand sides of the system as follows:

```
function f = lorenz(t, x)
f = zeros(3,1);
f(1) = 10 * (x(2) - x(1));
f(2) = -x(1) * x(3) + 28 * x(1) - x(2);
f(3) = x(1) * x(2) - 8 * x(3) / 3;
```

The three elements of the MATLAB vector `x`—`x(1)`, `x(2)`, and `x(3)`—respectively represent the three dependent scalar variables x, y, and z. The elements of the vector `f` represent the right-hand sides of the three DEs. When a vector is returned by such a DE function, it must be a column vector—hence, the statement

```
f = zeros(3,1);
```

Step 2 Use the following commands to solve the system from, say, $t = 0$ to $t = 10$. Note that we use `ode45` now, since it is more accurate.

```
x0 = [-2 -3.5 21];   % initial values in a vector
[t, x] = ode45(@lorenz, [0 10], x0);
plot(t,x)
```

You will see three graphs, for x, y, and z (in different colors).

Step 3 It is easier to see the effect of changing the initial values if there is only one graph in the figure to start with. It is in fact best to plot the solution $y(t)$ on its own. The MATLAB solution `x` is actually a matrix with three columns (as you can see from `whos`). The solution $y(t)$ that we want will be the second column, so plot it by itself using the command

```
plot(t,x(:,2),'g')
```

Then keep the graph on the axes with the command `hold`.

Now we can see the effect of changing the initial values. We just change the initial value of $x(0)$ from -2 to -2.04 (a change of only 2% and in only one of the three initial values). The following commands will do this, solve the ODEs, and plot the new graph of $y(t)$ (in a different color):

```
x0 = [-2.04 -3.5 21];
[t, x] = ode45(@lorenz, [0 10], x0);
plot(t,x(:,2),'r')
```

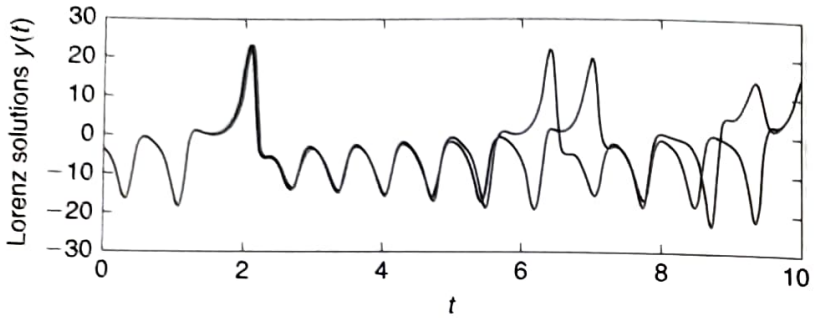


FIGURE 17.6
Chaos?

Figure 17.6 shows that the two graphs are practically indistinguishable until t is about 1.5. The discrepancy is gradual until t reaches about 6, when the solutions suddenly and shockingly flip over in opposite directions. As t increases further, the new solution bears no resemblance to the old one.

Now solve the system (17.12)–(17.14) with the original initial values using `ode23` this time:

```
x0 = [-2 -3.5 21];
[t,x] = ode23(@lorenz, [0 10], x0);
```

Plot the graph of $y(t)$ only—that is, `x(:,2)`—and then superimpose the `ode45` solution *with the same initial values* (in a different color). A strange thing happens—the solutions begin to deviate wildly for $t > 1.5$. The initial conditions are the same; the only difference is the order of the Runge-Kutta method.

Finally solve the system with `ode23s` and superimpose the solution. (`s` stands for "stiff." For a stiff ODE, solutions can change on a very short time scale compared to the interval of integration.) The `ode45` and `ode23s` solutions start to diverge only at $t > 5$. The explanation is that `ode23`, `ode23s`, and `ode45` all have numerical inaccuracies (if we could compare them with the exact solution—which incidentally can't be found). However, the numerical inaccuracies are *different* in the three cases and have the same effect as starting the numerical solution with slightly different initial values.

How do we know when we have the "right" numerical solution? We don't. The best we can do is increase the accuracy of the numerical method until no further wild changes occur over the interval of interest. In our example, then, we can only be pretty sure of the solution for $t < 5$ (using `ode23s` or `ode45`). If that's not good enough, we have to find a more accurate ODE solver.

Beware: chaotic DEs are very tricky to solve.

Incidentally, if you want to see the famous "butterfly" representation of chaos, just plot x against z as time increases (the resulting graph is called a *phase plane plot*). The following command will do the trick:

```
plot(x(:,1), x(:,3))
```

What you will see is a static 2-D projection of the *trajectory*, that is, the solution developing in time. **Demos** in the MATLAB Launch Pad includes an example that enables you to see the trajectory evolving dynamically in 3D (**Demos: Graphics: Lorenz attractor animation**).

17.6.3 Passing additional parameters to an ODE solver

In the examples of MATLAB ODE solvers, all of the coefficients on the right-hand sides (e.g., the value 28 in Equation (17.13)) are constants. In a real modeling situation, you will most likely want to change such coefficients frequently. To avoid having to edit the function files each time you do so, pass the coefficients as additional parameters to the ODE solver, which in turn passes them to the DE function. To see how this is done, consider the Lotka-Volterra *predator-prey* model:

$$dx/dt = px - qxy \tag{17.15}$$

$$dy/dt = rxy - sy \tag{17.16}$$

where $x(t)$ and $y(t)$ are the prey and predator population sizes at time t, and p, q, r, and s are biologically determined parameters. For this example, we take $p = 0.4$, $q = 0.04$, $r = 0.02$, $s = 2$, $x(0) = 105$, and $y(0) = 8$.

First write a function M-file, `volterra.m`, as follows:

```
function f = volterra(t, x, p, q, r, s)
f = zeros(2,1);
f(1) = p*x(1) - q*x(1)*x(2);
f(2) = r*x(1)*x(2) - s*x(2);
```

Then enter the following statements in the Command Window, which generate the characteristically oscillating graphs in Figure 17.7:

```
p = 0.4; q = 0.04; r = 0.02; s = 2;
[t,x] = ode23(@volterra,[0 10],[105; 8],[],p,q,r,s);
plot(t, x)
```

Note that the additional parameters (`p`, `q`, `r`, and `s`) have to follow the fourth input argument of the ODE solver, `options` (see `Help`). If no options have been set (as in our case), use `[]` as a placeholder for the `options` parameter.

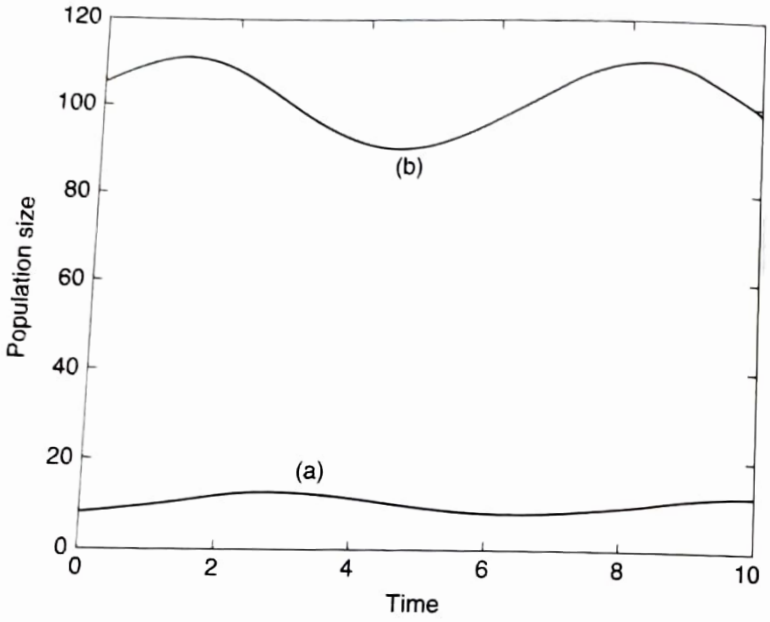


FIGURE 17.7

Lotka-Volterra model: (a) predator curve; (b) prey curve.

You can now change the coefficients from the Command Window and get a new solution, without editing the function file.

17.7 A PARTIAL DIFFERENTIAL EQUATION

The numerical solution to partial differential equations (PDEs) is a vast subject beyond the scope of this book. However, a class of PDEs called *parabolic*, often lead to solutions in terms of sparse matrices, which were mentioned briefly in Chapter 16. One such example is considered in this section.

17.7.1 Heat conduction

The conduction of heat along a thin uniform rod may be modeled by the partial differential equation

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} \tag{17.17}$$

where $u(x, t)$ is the temperature distribution a distance x from one end of the rod at time t, assuming that no heat is lost from the rod along its length.

Half the battle in solving PDEs is mastering the notation. We set up a rectangular grid with step-lengths of h and k in the x and t directions, respectively. A general point on the grid has coordinates $x_i = ih$, $y_j = jk$. A concise notation for $u(x, t)$ at x_i, y_j is thus $u_{i,j}$.